# Lesson 13 Teacher's Script

A Hit That Reacts

VIDEO GAME CREATION

## [0:00-0:05] Welcome Back

"Welcome back! Last week your game got its voice. This week, it gets a body - a hit is about to actually LOOK like a hit, not just sound like one."

"Since Lesson 4, an enemy you shoot has just... vanished. Functional, but not satisfying. Today we fix that: a brief flash, then a small explosion, before it disappears."

## [0:05-0:15] A Hit That Reacts: Two New Visual Touches

"Two things are happening today, and they're different kinds of things. First: the enemy itself briefly flashes - a reaction ON the thing that got hit. Second: a completely separate explosion effect appears at that spot and plays itself out - independent of the enemy entirely."

"Here's the rule for today, and it's important: your score, your lives, your wave counter, your win and lose logic - none of that changes WHEN it happens. Only how the enemy's removal LOOKS is changing. If you find yourself moving scoring code around, stop and re-check the Handout."

*[Timing checkpoint: wrap by 0:15]*

## [0:15-0:40] Pausing Mid-Script: Three Engine Families, Three Solutions

[Demo cue: switch to Scratch, open a project with Lesson 12's four sounds working]

"Here's the actual challenge this week: to see a flash, the enemy has to stick around for a fraction of a second AFTER being hit, before it's actually removed. That means something in our code needs to pause - just for a moment - without freezing the whole game."

"Scratch actually has the simplest answer of any engine in this class: a plain 'wait' block. Watch - right in the Enemy's touching-Bullet branch, after scoring and the sound, I set the color effect to flash it white, then just drop in 'wait 0.1 seconds', then move the existing win-check and delete-this-clone below that wait." [Build it live, demonstrate the flash-then-disappear on a hit] "Everything below that wait block just... happens a moment later. No special keyword needed - Scratch was built from the ground up to run every sprite's scripts side by side, so pausing one of them was never a hard problem to solve."

"Godot and Unity get the same result with a bit more ceremony - GDScript's await keyword and C#'s Coroutines both pause a script mid-line, the same idea as Scratch's wait, just spelled with more setup. But GameMaker and TIC-80 CAN'T do this at all - their code can't pause mid-script, full stop. So they solve it differently: GameMaker sets an Alarm and finishes the job in a second

event, a fraction of a second later. TIC-80 does the Lua version of the same trick with a countdown variable. Four engines, two genuinely different families of solution to the same problem - and that's worth understanding, not just copying."

*[Timing checkpoint: wrap by 0:40]*

## [0:40-0:50] An Explosion That Minds Its Own Business

"Now the easier half: the explosion effect. It spawns at the hit location and IMMEDIATELY forgets about everything else - it doesn't know your score, doesn't know your wave, doesn't even know which enemy made it. It just plays itself out and cleans itself up, on its own."

"This should feel familiar - it's the exact same discipline your bullets have followed since Lesson 3, and your enemies since Lesson 5. Every object in this game is responsible for exactly one job, and cleans up after itself. Today's explosion is just the newest member of that club."

*[Timing checkpoint: wrap by 0:50]*

## [0:50-0:55] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one. One new asset is provided: a small explosion image. Unreal folks, you're using your existing Cube/Sphere shapes instead; TIC-80 folks, you're drawing your own tiny explosion sprite."

[Hand out engine-specific handouts]

"Test a FULL round when you're done - shooting, a wave clear, even a loss - and confirm nothing about your SCORING changed, only the visuals. That's today's real Level 2 check."

"Questions before we start lab? ... Let's make getting hit feel like something."

## Lab Time Notes

- The most common mistake this week: accidentally moving score/win-check logic along with the delay, instead of leaving it immediate and only delaying the enemy's final removal. Watch for this specifically.
- GameMaker and TIC-80 students will need a bit more circulating time this week - splitting logic across two events/table-fields is a bigger structural change than most weeks. Budget extra minutes for these clusters.
- 001 Game Creator: remind students the enemy's own flash may be brief or inconsistent in their installed version - that's expected and documented, not something to debug endlessly. The explosion effect is the reliable part.
- This is a fun, visual week - encourage students to actually watch a few hits happen slowly rather than just confirming it "works," and consider a quick show-and-tell of favorite explosion tweaks during the share-out.
- If a cluster finishes Level 1 quickly, point them at Level 3's animation-timing ideas (fading flash, scaling explosion, multi-frame flicker) rather than rushing ahead to Lesson 14's material.